

BÁO CÁO DỰ ÁN LẬP TRÌNH

Đề tài: Hệ thống Đấu giá Trực tuyến (UET Auction House)

Danh sách nhóm sinh viên thực hiện:

STT	Mã Sinh Viên	Họ và Tên	Tỷ lệ đóng góp
1	25021980	Đào Đình Tiến	25%
2	25022039	Nguyễn Văn Trường	25%
3	25022032	Nguyễn Đức Trung	25%
4	25021874	Ngô Quang Minh	25%

1. Giới thiệu mục tiêu và phạm vi thực hiện

Dự án **UET Auction House** được phát triển nhằm mục tiêu xây dựng một nền tảng phần mềm đấu giá trực tuyến (bidding) hoàn chỉnh, cho phép nhiều người dùng cùng tham gia cạnh tranh giá để mua sản phẩm trong một khoảng thời gian xác định. Hệ thống được thiết kế theo kiến trúc Client-Server, đảm bảo khả năng xử lý thời gian thực, tính ổn định và tính bảo mật trong quá trình giao dịch.

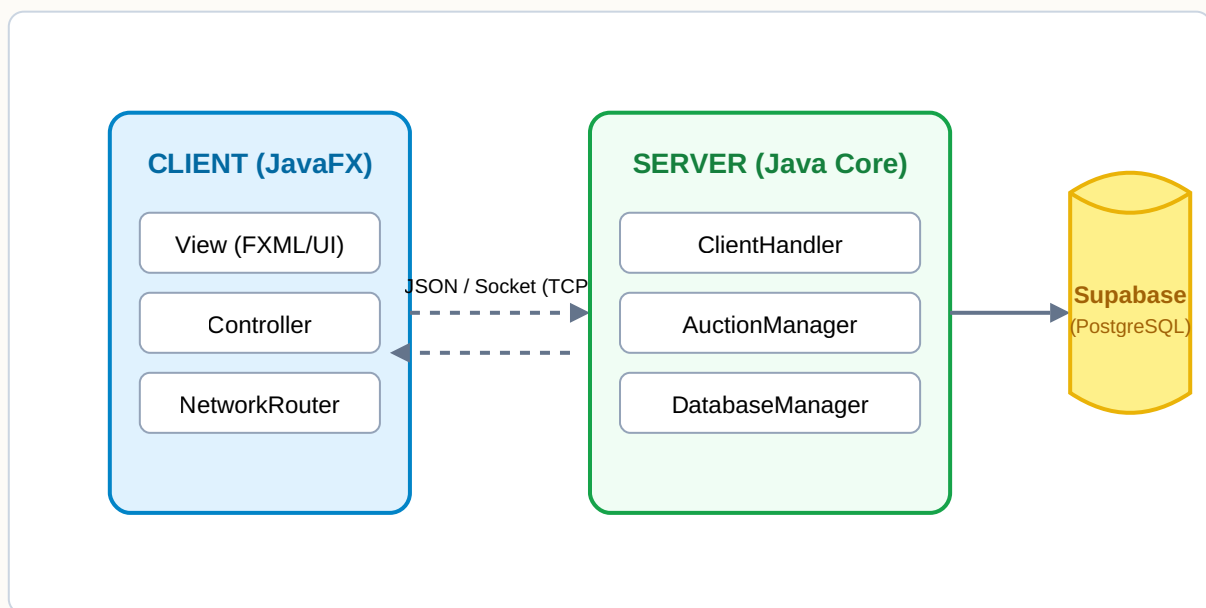
Phạm vi thực hiện của dự án bao gồm:

- Xây dựng ứng dụng Client Desktop bằng JavaFX, cung cấp giao diện trực quan cho cả người đấu giá (Bidder), người bán (Seller) và Quản trị viên (Admin).
- Phát triển Server trung tâm bằng Java Socket, xử lý logic nghiệp vụ đấu giá, quản lý phiên kết nối và tương tác với cơ sở dữ liệu.

- Tích hợp cơ sở dữ liệu đám mây Supabase (PostgreSQL) và hệ thống lưu trữ ảnh (Supabase Storage).
- Áp dụng các kỹ thuật lập trình hướng đối tượng (OOP), lập trình đa luồng (Multithreading) và các mẫu thiết kế (Design Patterns) như Observer, Singleton.

2. Kiến trúc tổng thể của hệ thống

Hệ thống UET Auction House được xây dựng theo kiến trúc **Client-Server phân tầng (MVC)**, đảm bảo tính tách biệt giữa giao diện, nghiệp vụ và lưu trữ dữ liệu.



Mô tả kiến trúc:

- **Tầng Client (Giao diện):** Áp dụng mô hình MVC với JavaFX. Giao diện (View) được thiết kế qua FXML, logic điều khiển nằm trong các Controller. Lớp **NetworkRouter** đóng vai trò trung gian, tiếp nhận luồng dữ liệu JSON từ Server và cập nhật giao diện thông qua **Platform.runLater()**.
- **Tầng Server (Xử lý nghiệp vụ):** Hoạt động như một máy chủ TCP/IP. Mỗi kết nối Client được giao cho một luồng **ClientHandler** riêng biệt. Lớp **AuctionManager** quản lý tập trung các phiên đấu giá đang diễn ra, sử dụng khóa (Lock) để xử lý tương tranh.
- **Tầng Dữ liệu (Database):** Chỉ Server mới được phép truy cập cơ sở dữ liệu Supabase thông qua **DatabaseManager**, sử dụng JDBC để thực thi SQL và quản lý transaction (ACID).

3. Các chức năng đạt được và hướng giải quyết

Chức năng	Hướng giải quyết / Triển khai	Lý do lựa chọn / Đánh giá
Thiết kế OOP & Design Pattern	- Cây kế thừa: <code>User</code> -> <code>Admin</code>, <code>Member</code>. <code>Items</code>, <code>AuctionSession</code>. - Singleton: <code>AuctionManager</code>, <code>DatabaseManager</code>. - Observer: Lớp <code>AuctionObserver</code> để push update giá.	Tuân thủ tính đa hình, dễ mở rộng vai trò người dùng. Singleton đảm bảo luồng quản lý tập trung, tránh rò rỉ bộ nhớ.
Xử lý đấu giá đồng thời (Concurrency)	Sử dụng <code>ReentrantLock</code> trong <code>AuctionSession</code> khóa luồng cục bộ từng phiên. Dùng <code>FOR UPDATE</code> trong SQL (Pessimistic Locking) tại <code>executeSafeBidTransaction()</code>.	Ngăn chặn triệt để tình trạng Race Condition, Lost Update. Khóa cục bộ theo phiên giúp Server vẫn xử lý mượt mà các phiên đấu giá khác.
Realtime Update (Observer)	Tích hợp cơ chế Broadcast từ <code>AuctionManager</code>. Khi có Bid mới, Server đẩy JSON <code>LIVE_BID_UPDATE</code> đến tất cả <code>ClientHandler</code> đang lắng nghe.	Cập nhật giao diện lập tức không cần polling (tải lại trang), tiết kiệm tài nguyên mạng và băng thông.
Gia hạn phiên (Anti-sniping)	Trong <code>processIncomingBid()</code>, tính toán <code>ChronoUnit.SECONDS</code> giữa hiện tại và <code>endTime</code>. Nếu < 120s, cộng thêm 1 phút và cập nhật DB.	Mang lại sự công bằng, chặn chiến thuật bắn tĩa giây cuối, bám sát nghiệp vụ thực tế của các sàn đấu giá chuyên nghiệp.
Auto-Bidding (Đấu giá tự động)	Lưu <code>max_budget</code> vào DB. Khi có người bid, hàm <code>executeAutoBidCounterTransaction()</code> chạy vòng lặp <code>do-while</code> để tạo "Bid War" tự động.	Giúp người dùng không cần canh màn hình liên tục. Logic xử lý ngay tại Server đảm bảo tốc độ phản hồi tính bằng mili-giây.
Bid History Visualization (Biểu đồ)	Sử dụng <code>LineChart</code> (JavaFX). Mỗi khi nhận Broadcast giá mới, tự động append <code>XYChart.Data</code> theo <code>LocalTime.now()</code>.	Vẽ biểu đồ đường cong giá realtime trực quan. Tắt hiệu ứng <code>setAnimated(false)</code> để

tránh bug dồn điểm của
JavaFX.

4. Phân chia công việc giữa các thành viên

Nhóm đã sử dụng Git/GitHub để quản lý mã nguồn và thống nhất phân chia công việc theo tỷ lệ **25%** cho mỗi thành viên dựa trên khối lượng và độ khó của công việc:

Thành viên	Nhiệm vụ chi tiết	Đánh giá
Đào Đình Tiến (25021980)	- Thiết kế giao diện cơ bản (Login, SignUp, Home). - Xây dựng <code>NetworkRouter</code> chuyển đổi, parse JSON phía Client. - Phân quyền giao diện Admin/User.	25%
Nguyễn Văn Trường (25022039)	- Thiết kế UI phức tạp (Bidding, Dashboard, Notification). - Tích hợp biểu đồ LineChart, hiệu ứng Skeleton Loading. - Viết logic cập nhật UI từ Background Thread (<code>Platform.runLater</code>).	25%
Nguyễn Đức Trung (25022032)	- Thiết kế Data Model (User, Admin, Item, Auction, Bid). - Lập trình tính năng Auto-Bidding và thuật toán Anti-Sniping. - Xử lý nén ảnh Base64 và API Supabase Storage.	25%
Ngô Quang Minh (25021874)	- Lập trình mạng (Socket Client/Server, Thread Pool). - Quản lý DatabaseManager (JDBC, ACID Transactions). - Xử lý đồng bộ hóa (Concurrency) bằng <code>ReentrantLock</code> và SQL <code>FOR UPDATE</code>.	25%